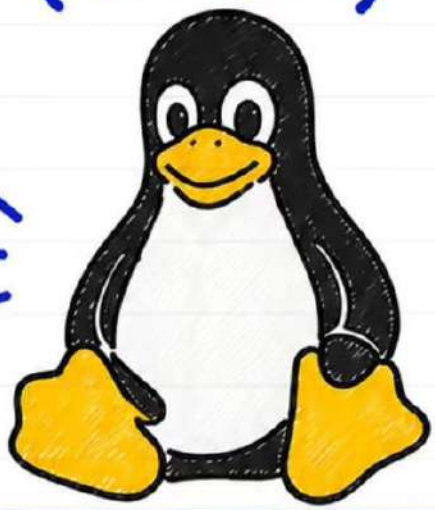
@qa_insights

LINUX

COMPLETE NOTES

(Linux)



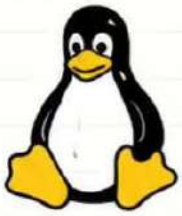
BEGINNER TO ADVANCED

SWIPE TO NEXT →

FOLLOW | SHARE | COMMENT | LIKE

1. INTRODUCTION TO LINUX

"Linux is a free and open-source operating system based on the Unix kernel. It is secure, stable, and highly customizable."



HISTORY

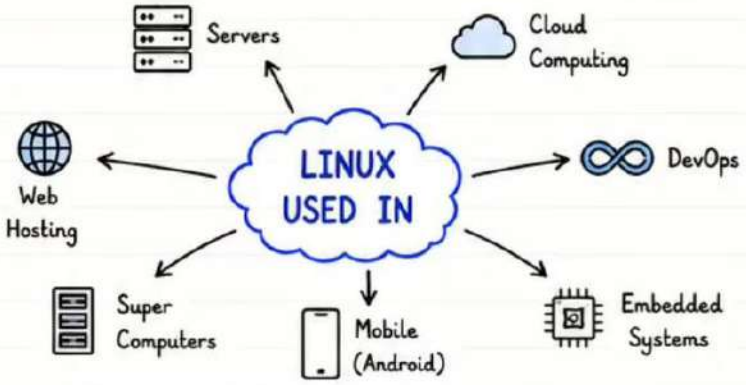
- Created by Linus Torvalds in 1991.
- Inspired by Unix and developed as a hobby project.
- Released under the GNU General Public License (GPL).
- Today, Linux powers servers, desktops, cloud, mobile devices and more!

FEATURES

- ✓ Open Source – Free to use and modify.
- ✓ Multi-user – Multiple users can work simultaneously.
- ✓ Multi-tasking – Handles multiple tasks efficiently.
- ✓ Secure – Strong permissions and user management.
- ✓ Stable – Runs for long time without reboot.
- ✓ Customizable – Can be adapted to any need.

WHY LINUX?

- ✓ It is free and open-source.
- ✓ Highly secure and reliable.
- ✓ Preferred choice for servers & cloud.
- ✓ Supports a wide range of hardware.
- ✓ Strong community support.
- ✓ Ideal for developers, DevOps and QA.



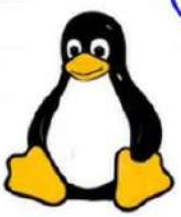
LINUX vs WINDOWS

BASIS	LINUX	WINDOWS
License	Open Source (Free)	Proprietary (Paid)
Cost	Free	Paid
Security	More secure, less vulnerable	More vulnerable to malware
Customization	Highly customizable	Limited customization
Performance	Lightweight, faster	Resource intensive
User Interface	Varies (Multiple Desktop Environments)	Graphical, user-friendly
Use Case	Servers, DevOps, Cloud, Development	Gaming, Office, Desktop use

REMEMBER

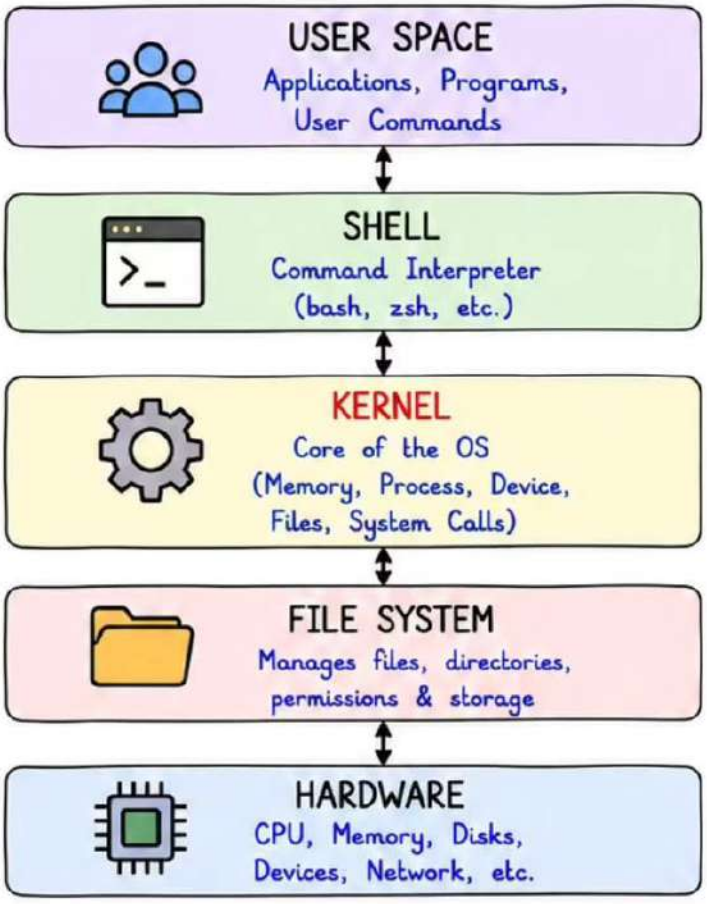
- ★ Linux is not just an OS, it's an ecosystem.
- ★ Most of the internet runs on Linux!

2. LINUX ARCHITECTURE





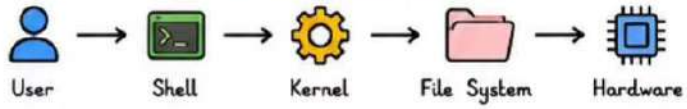
Linux follows a layered architecture where each layer plays a specific and important role.



LAYERS EXPLAINED

1. **USER SPACE**
   This layer contains all user applications, programs and commands.
2. **SHELL**
   Acts as an interface between the user and the kernel. It takes commands from the user and passes to the kernel.
3. **KERNEL**
   The heart of Linux. It manages system resources, handles system calls and acts as a bridge between software and hardware.
4. **FILE SYSTEM**
   Organizes and manages data on storage devices. It handles files, directories, permissions and disk space.
5. **HARDWARE**
   Physical components of the system like CPU, RAM, Disk, Network Cards, I/O Devices, etc.

HOW IT WORKS ?



User gives a command → Shell interprets it → Kernel processes it → File system reads/writes data → Hardware executes the task.

REMEMBER

- Everything in Linux is a file.
- Kernel is always in control.
- Without kernel, system cannot function.
- Shell is optional, but very powerful.

QUICK COMMANDS

```
uname -a          # System information
cat /proc/version # Kernel details
whoami            # Current user
ls /              # List root directory
ps aux            # Running processes
df -h             # Disk usage
free -m           # Memory usage
```

EXAMPLE: CHECK KERNEL VERSION

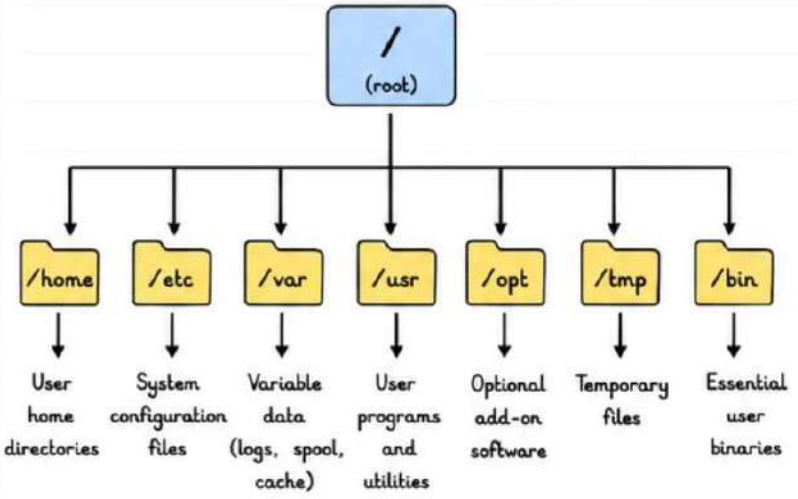
```
$ uname -r        # show kernel version
```

Output: 6.5.0-25-generic

3. LINUX FILE SYSTEM

Everything in Linux is a file.
Files are organized in a hierarchical tree structure starting from the root (/).

FILE SYSTEM HIERARCHY



EXAMPLE STRUCTURE

```
/
├── home
│   └── qa_user
├── etc
├── var
│   ├── log
│   └── cache
├── usr
│   ├── bin
│   └── local
├── opt
├── tmp
└── bin
```

TIP

- ✓ Root (/) is the base of the entire system.
- ✓ Linux is case-sensitive.
- ✓ Every file or directory inherits permissions from its parent.
- ✓ Use "/" in paths (not \\ like Windows).

IMPORTANT DIRECTORIES

/ (root)
The top of the directory tree. All files and directories start from here.

/home
Contains home directories for users. e.g., /home/qa_user

/etc
Stores system-wide configuration files. Important for system settings.

/var
Holds variable data like logs, spool files, cache, databases, mail, etc.

/usr
Contains most user programs, libraries, documentation and utilities.

/opt
Used to install optional or third-party software.

/tmp
Stores temporary files. Files may be deleted on reboot.

/bin
Contains essential command binaries needed for system operation.

QUICK COMMANDS

```
ls /            # List root directory
ls /etc         # List /etc directory
cd /home        # Go to /home
tree /          # Show directory tree (if installed)
```

REMEMBER

- ★ Root (/) is the starting point of everything.
- ★ Keep system directories safe. Do not modify files unless you know what you are doing.
- ★ Use meaningful names for files & directories.

4. NAVIGATION COMMANDS





Navigation commands help us move around the Linux file system and explore directories.

1 pwd

Print Working Directory

Shows the current directory you are in.

Example

```
$ pwd
/home/qa_user/projects
```

2 ls

List

Lists files and directories in the current directory.

Example

```
$ ls
file1.txt  docs  scripts  test
```

3 cd

Change Directory

Change your current directory to another.

Example

```
$ cd /home/qa_user/docs
$ pwd
/home/qa_user/docs
```

4 tree

Tree View

Displays directories in a tree like structure.

Example

```
$ tree
.
├── docs
│   ├── file1.txt
│   └── file2.txt
└── scripts
    └── run.sh
```

5 clear

Clear Screen

Clears the terminal screen for a fresh start.

Example

```
$ clear
# Screen will be cleared
```

6 history

Command History

Shows previously executed commands.

Example

```
$ history
1  ls
2  cd /home/qa_user
3  pwd
4  tree
5  clear
```

REAL EXAMPLE (PRACTICAL USAGE)

1	pwd	# see current location →	/home/qa_user
2	ls	# list files & folders →	docs scripts test notes.txt
3	cd docs	# go to docs folder →	–
4	pwd	# confirm new location →	/home/qa_user/docs
5	tree	# show tree structure →	. ├── file1.txt └── file2.txt
6	cd ..	# go back to parent →	/home/qa_user
7	history	# view command history →	1 ls | 2 cd docs | 3 pwd | 4 tree | 5 cd ..

REMEMBER

- Use Tab key for auto-completion.
- Use cd .. to move one level up.
- Use cd ~ to go to home directory.

5. FILE & DIRECTORY COMMANDS

These commands help us to create, manage, copy, move and view files & directories in Linux.

1 mkdir

Make Directory

Creates a new directory.

Example

```
$ mkdir myfolder
$ mkdir -p a/b/c
# -p creates parent
  directories if not exist
```

2 touch

Create File

Creates an empty file or updates timestamp.

Example

```
$ touch file.txt
$ touch a.txt b.txt
# create multiple files
```

3 cp

Copy Files or Directories

Copies files or directories from one place to another.

Example

```
$ cp file.txt backup.txt
$ cp -r myfolder newfolder
# -r for recursive copy
```

4 mv

Move or Rename

Moves files/directories or renames them.

Example

```
$ mv file.txt docs/
$ mv oldname.txt newname.txt
$ mv myfolder newfolder
# move/rename directory
```

5 rm

Remove Files

Deletes files.

Example

```
$ rm file.txt
$ rm a.txt b.txt
# delete multiple files
$ rm -r myfolder
# -r for recursive delete
```

6 rmdir

Remove Directory

Deletes empty directories only.

Example

```
$ rmdir emptydir
$ rmdir -p a/b/c
# remove parent dirs if
  empty
```

7 cat

Concatenate & View

Displays the content of a file on terminal.

Example

```
$ cat file.txt
$ cat file1.txt file2.txt
# view multiple files
```

8 nano

Nano Editor

Simple text editor in terminal.

Example

```
$ nano file.txt
# Ctrl + O -> Save
# Ctrl + X -> Exit
```

9 vi

Vi Editor

Powerful text editor (Advanced).

Example

```
$ vi file.txt
# i    -> Insert mode
# Esc  -> Exit insert
# :wq  -> Save & Exit
# :q!  -> Quit without save
```

COMMON OPTIONS

-r	Use with cp, rm	# Recursive
-p	Use with mkdir, rmdir	# Parent directories
-f	Force operation	# Do without confirmation
-i	Interactive mode	# Ask before overwrite/delete
-v	Verbose mode	# Show detailed output

REMEMBER

- Be careful with rm -r (data loss).
- Use cp -r to copy directories.
- nano is beginner friendly.
- vi is powerful but has a learning curve.
- Always check before deleting!

6. FILE PERMISSIONS

Linux uses permissions to control who can read, write and execute files & directories.

1) WHO OWNS THE FILE ?

USER (Owner)
The owner of the file.
Has full control over the file.

GROUP
A set of users. Members share the same group permissions.

OTHERS
All other users on the system who are not the owner or in the group.

2) PERMISSION TYPES

	Value
READ (r) Allows viewing the contents.	4
WRITE (w) Allows modifying or deleting.	2
EXECUTE (x) Allows executing the file or accessing directory.	1

3) PERMISSION TABLE (rwx)

Permission	Symbol	Numeric	Meaning
Read	r	4	View file contents
Write	w	2	Modify file contents
Execute	x	1	Run file / Enter directory
Read + Write	rw	6	4 + 2
Read + Execute	rx	5	4 + 1
Write + Execute	wx	3	2 + 1
Read + Write + Execute	rwx	7	4 + 2 + 1
No Permission	---	0	No access

4) CHMOD COMMAND

Used to change permissions.

Syntax (Symbolic)

```
$ chmod [who] [operation] [permission] file
```

- who : u (user) , g (group) , o (others) , a (all)
- operation : + (add) , - (remove) , = (set)
- permission : r , w , x

Examples

```
$ chmod u+x script.sh      # add execute for user
$ chmod g-w file.txt       # remove write for group
$ chmod o=r file.txt       # set read for others
$ chmod a=rwx myfile       # full access for all
```

5) CHOWN COMMAND

Used to change owner and group of a file/directory.

Syntax

```
$ chown [user] file
$ chown [user]:[group] file
$ chown -R [user]:[group] directory  # recursive
```

Examples

```
$ chown qa_user file.txt              # change owner
$ chown qa_user:qa_group file.txt     # change owner and group
$ chown -R qa_user:qa_group project/  # recursive change
```

6) HOW TO VIEW PERMISSIONS ?

Use ls -l command

```
$ ls -l file.txt
-rwxr--r--  1 qa_user qa_group  1024 May 25 10:30 file.txt
```

Permissions | Owner | Group | Size | Date | Name

REMEMBER

- Files have an owner and a group.
- Use least privilege principle.
- Be careful with 777 (full access).
- Always check before changing permissions.

7. SEARCHING & TEXT PROCESSING

These commands help us search for files and text, and process data efficiently in Linux.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	find	Search for files and directories.	`$ find /home -name "file.txt"`	Finds file.txt in /home
2	locate	Find files using pre-built database.	`$ locate file.txt`	Fast file search using index
3	grep	Search for pattern in files.	`$ grep "error" app.log`	Finds lines containing "error"
4	awk	Pattern scanning and processing language.	`$ awk '{print $1}' data.txt`	Prints first column from data.txt
5	sed	Stream editor for filtering & transforming.	`$ sed 's/old/new/g' file.txt`	Replaces old with new in file.txt
6	sort	Sort lines of text files.	`$ sort names.txt`	Sorts names in alphabetical order
7	uniq	Remove duplicate lines.	`$ uniq list.txt`	Shows only unique lines
8	wc	Count lines, words and characters.	`$ wc file.txt`	Counts lines, words and characters
9	head	Display first few lines of a file.	`$ head -n 10 file.txt`	Shows first 10 lines
10	tail	Display last few lines of a file.	`$ tail -n 10 file.txt`	Shows last 10 lines

SAMPLE WORKFLOW

1. Find all log files
   `$ find /var/log -name "*.log"`
2. Search for "error"
   `$ grep "error" app.log`
3. Sort and remove duplicates
   `$ grep "error" app.log | sort | uniq`
4. Count total lines
   `$ wc -l app.log`

EXAMPLE: error.log

```
2024-05-25 ERROR Failed to start
2024-05-25 INFO Service running
2024-05-25 ERROR Connection lost
2024-05-25 WARN Low memory
2024-05-25 ERROR Timeout occurred
```

```
$ grep "ERROR" error.log
2024-05-25 ERROR Failed to start
2024-05-25 ERROR Connection lost
2024-05-25 ERROR Timeout occurred
```

REMEMBER

- grep is case-sensitive.
- locate is faster but may be outdated.
- Use | (pipe) to combine commands.
- Always read man page (man command) for options.



8. PROCESS MANAGEMENT



These commands help us view, control and manage processes running in a Linux system.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	ps	Report snapshot of current processes.	$ ps aux	Shows currently running processes.
2	top	Real-time view of processes.	$ top	Displays dynamic process info.
3	htop	Interactive process viewer (better than top).	$ htop	User-friendly real-time process monitor.
4	jobs	List background and stopped jobs.	$ jobs	Shows jobs started from current shell.
5	bg	Resume a stopped job in background.	$ bg %1	Continues job in background.
6	fg	Bring a background job to foreground.	$ fg %1	Brings job to current terminal.
7	kill	Terminate a process using PID.	$ kill 1234	Sends TERM signal (default).
8	kill -9	Force kill a process (SIGKILL).	$ kill -9 1234	Forcefully terminates unresponsive process.
9	nice	Run a process with modified priority.	$ nice -n 10 command	Adjusts process priority (lower value = higher priority).

EXAMPLES

```
# View all running processes
$ ps aux
# Real-time process monitoring
$ top
# List current jobs
$ jobs
# Resume job 1 in background
$ bg %1
# Bring job 1 to foreground
$ fg %1
# Kill process with PID 1234
$ kill 1234
# Force kill process with PID 1234
$ kill -9 1234
# Run process with low priority
$ nice -n 10 sort largefile.txt
```

JOB STATES

- `o` Running in foreground
- `&` Running in background
- `T` Stopped (by Ctrl+Z)
- `+` Most recent job
- `-` Second most recent job

Example:

```
$ sleep 100 &
[1] 21567
$ jobs
[1]+ Running      sleep 100 &
[2]- Stopped      vim file.txt
```

REMEMBER

- Use ps aux for detailed process info.
- top is useful but htop is easier to use.
- Use Ctrl + Z to stop a process and bg to run in background.
- kill sends default TERM signal (kind request).
- kill -9 should be used only when process does not stop.
- nice command starts process with lower priority.

9. DISK & MEMORY COMMANDS



These commands help us monitor disk usage, memory usage and system performance.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	df	Disk free space report.	$ df -h	Shows disk space usage of file systems.
2	du	Estimate file and directory space usage.	$ du -sh /home/user	Shows size of files or directories.
3	free	Display memory usage.	$ free -h	Shows total, used and free memory.
4	uptime	Show how long system has been running.	$ uptime	Shows system uptime and load average.
5	vmstat	Report virtual memory statistics.	$ vmstat 1 5	Shows processes, memory, CPU and I/O stats.
6	iostat	I/O statistics for CPU and devices.	$ iostat -xz 1 5	Shows CPU and device I/O performance.
7	mount	Mount a file system.	$ mount /dev/sdb1 /mnt	Attaches a filesystem to a directory.
8	umount	Unmount a file system.	$ umount /mnt	Detaches a filesystem from a directory.
9	lsblk	List block devices.	$ lsblk	Shows information about block devices.
10	df -i	Show inode usage.	$ df -i	Shows inode usage of file systems.

EXAMPLES

```
$ df -h
  # disk usage in human readable
$ du -sh /var/log
  # size of /var/log
$ free -h
  # memory usage
$ uptime
  # system uptime
$ vmstat 1 5
  # report every 1 sec, 5 times
$ iostat -xz 1 5
  # CPU and device /dev/sdb1 /mnt
$ mount /mnt
  # unmount device
```

SAMPLE OUTPUTS

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   20G   28G  42% /
/dev/sdb1       100G   60G   40G  60% /data
```

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7G        2.1G        2.3G        0.3G        3.3G        4.8G
Swap:          2.0G          0B        2.0G
```

```
$ uptime
14:35:21 up 3 days,  2:45,  2 users,
load average: 0.08, 0.12, 0.15
```

REMEMBER

- Use df -h to check disk space regularly.
- Use free -h to monitor memory usage.
- High load average (>1) means system is busy.
- Use vmstat and iostat for performance analysis.
- Always unmount before removing devices.

TIP Combine commands with pipes for powerful reports!

```
$ df -h | grep sda    # filter output
$ du -hs * | sort -h  # sort by size
```

10. NETWORKING COMMANDS

These commands help us troubleshoot, test connectivity and manage network connections.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	ping	Test connectivity between hosts.	$ ping google.com	Checks if host is reachable and measures latency.
2	curl	Transfer data to/from a server.	$ curl -I https://example.com	Fetches data or HTTP headers from URL.
3	wget	Download files from the web.	$ wget https://file.com/file.zip	Downloads files non-interactively.
4	ssh	Securely connect to a remote server.	$ ssh user@server.com	Opens secure remote shell session.
5	scp	Securely copy files between hosts.	$ scp file.txt user@host:/path	Copies files over SSH securely.
6	netstat	Show network connections, routing and interfaces.	$ netstat -tulnp	Displays active connections and listening ports.
7	ss	Modern replacement for netstat.	$ ss -tulnp	Faster way to show sockets and connections.
8	nslookup	Query DNS to find IP or domain info.	$ nslookup example.com	Looks up DNS records (IP, MX, etc.).
9	dig	Advanced DNS lookup tool.	$ dig example.com	Detailed DNS query and troubleshooting.
10	traceroute	Trace the path packets take to a host.	$ traceroute google.com	Shows each hop to the destination.

EXAMPLES

```
$ ping -c 4 google.com
  # send 4 packets
$ curl -L https://example.com
  # follow redirects
$ wget -c http://file.com/file.zip
  # continue download
$ ssh -p 22 user@192.168.1.10
  # connect on port 22
$ scp report.pdf user@server:/home/user/
  # copy file to remote server
$ ss -tulnp | grep 80
  # check process on port 80
$ dig +short example.com
  # get only IP address
```

COMMON PORTS

PORT	SERVICE
20 / 21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
443	HTTPS
3306	MySQL

REMEMBER

- ping checks connectivity, not speed.
- curl is great for APIs and headers.
- Use ssh and scp for secure file transfers.
- ss is faster and more powerful than netstat.
- dig is more detailed than nslookup.
- traceroute helps find network issues (hops where it fails).

TIP Tools like ping, traceroute, ss and dig are super helpful for DevOps & QA engineers!



11. PACKAGE MANAGEMENT



These commands help us install, update, remove and manage software packages in Linux.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	apt	Advanced Package Tool (Debian/Ubuntu).	$ apt search nginx	Package manager for Debian based systems.
2	yum	Yellowdog Updater Modified (RHEL/CentOS).	$ yum list installed	Package manager for RHEL/CentOS (v7 and below).
3	dnf	Dandified YUM (Fedora/RHEL 8+).	$ dnf repolist	Next-gen package manager (Fedora, RHEL 8+).
4	rpm	Red Hat Package Manager.	$ rpm -qa | grep httpd	Manages RPM packages (install, query, verify).
5	dpkg	Debian Package Manager.	$ dpkg -l | grep vim	Manages DEB packages on Debian based systems.
6	install	Install a package.	$ apt install nginx	Installs a software package.
7	update	Update package lists or upgrade packages.	$ yum update	Updates package lists or installed packages.
8	remove	Remove a package.	$ apt remove nginx	Removes a package from the system.

PACKAGE MANAGERS

- Debian / Ubuntu — apt, dpkg
- RHEL / CentOS 7 — yum, rpm
- Fedora / RHEL 8+ — dnf, rpm

COMMON OPERATIONS EXAMPLES

APT (Debian/Ubuntu)

- $ sudo apt update
  # update package lists
- $ sudo apt install nginx
  # install package
- $ sudo apt upgrade
  # upgrade installed packages
- $ sudo apt remove nginx
  # remove package
- $ apt search nginx
  # search package

YUM/DNF (RHEL/CentOS/Fedora)

- $ sudo yum repolist
  # list repositories
- $ sudo yum install nginx
  # install package
- $ sudo yum update
  # update packages
- $ sudo yum remove nginx
  # remove package
- $ yum list installed
  # list installed packages

TIPS

- Use sudo with package commands.
- Run update before install/upgrade.
- Know your distro to use the right tool.
- Avoid using yum and dnf together on the same system.

RPM vs DEB

RPM

- Used in RHEL, CentOS, Fedora.
- File extension .rpm

DEB

- Used in Debian, Ubuntu.
- File extension .deb

REMEMBER

- apt = Debian / Ubuntu
- yum = RHEL / CentOS 7
- dnf = Fedora / RHEL 8+
- rpm & dpkg work on package files directly.

TIP Package management keeps your system secure, updated and stable! 😊



12. SHELL SCRIPTING BASICS



Shell scripting lets us automate tasks by writing commands in a file and executing them.

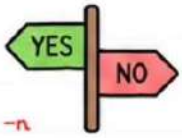
1 VARIABLES

- Used to store data.
- No spaces around = sign.
- Access with $ before name.

```
x = 10
name = QA
```

```
# variable example
name="QA_Insights"
count=5
echo "Hello $name, count is $count"
```

2 IF CONDITION



- Used for decision making.
- Common operators: -eq, -ne, -gt, -lt, -z, -n

```
read -p "Enter a number: " num
if [ $num -gt 10 ]; then
    echo "Number is greater than 10"
else
    echo "Number is 10 or less"
fi
```

3 LOOPS



- for loop: iterate over items
- while loop: repeat while condition is true

```
# for loop
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
# while loop
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    count=$((count+1))
done
```

4 FUNCTIONS



- Group commands into reusable blocks.
- Call by function name.

```
greet() {
    echo "Hello $1! Welcome to Shell Scripting."
}

greet "QA_Insights"    # function call
```

5 ARGUMENTS



- $0 = script name
- $1, $2, ... = positional arguments
- $# = total number of arguments

```
# arguments example
echo "Script: $0"
echo "First Arg: $1"
echo "Second Arg: $2"
echo "Total Args: $#"
```

REMEMBER

- Give execute permission: chmod +x script.sh
- Run script: ./script.sh
- Use # for comments
- Use meaningful variable and function names

6 SIMPLE SCRIPT EXAMPLE

Task: Take user name, show date, and list files in current directory

```
1  #!/bin/bash
2  # Simple script example
3
4  read -p "Enter your name: " user      } Take user input
5  echo "Hello $user!"
6
7  echo "Today's date and time: "        } Show date and time
8  date
9
10 echo "Files in current directory:"    } List files
11 ls -l
12
13 echo "Script execution completed."    } End message
14 exit 0
```

Run: ./myscript.sh



TIP Start small, test often, and build useful automation!





13. LINUX IN REAL PROJECTS

Linux is the backbone of modern IT infrastructure.
It powers tools, servers, pipelines and platforms.

WHERE IS LINUX USED?

1 SOFTWARE DEVELOPMENT

- Preferred OS for developers.
- Supports tools like Git, Vim, GCC, Make, Node.js, Python.
- Easy package management and environment setup.

2 CLOUD DEVOPS

- Linux servers run on AWS, Azure, GCP.
- Used in CI/CD, monitoring, logging and scripting.
- Core for Docker, Kubernetes and Terraform.

3 CYBER SECURITY

- Used by security pros (Kali Linux, Parrot OS).
- Essential for penetration testing and forensics.
- Tools: Nmap, Wireshark, Metasploit, Burp Suite.

4 DATA ENGINEERING

- Linux handles big data workloads.
- Runs Hadoop, Spark, Kafka, Airflow, etc.
- Stable, secure and great for processing large datasets.

5 AI / ML

- Linux is the go-to OS for AI/ML engineers.
- Supports Python, TensorFlow, PyTorch, CUDA, etc.
- Works seamlessly on GPUs and HPC clusters.

6 WEB HOSTING

- Most web servers run on Linux.
- Hosts Nginx, Apache, MySQL, PHP, Node.js apps.
- Reliable, lightweight and cost-effective.

REAL PROJECT WORKFLOW

DEVELOPER → **GIT** → **LINUX SERVER** → **DOCKER** → **JENKINS** → **KUBERNETES** → **CLOUD**

- **DEVELOPER**: Writes code and tests locally.
- **GIT**: Code is committed and pushed to repository.
- **LINUX SERVER**: Code is pulled on Linux server for build & execution.
- **DOCKER**: Application is containerized using Docker.
- **JENKINS**: Jenkins builds, tests and creates artifacts.
- **KUBERNETES**: K8s deploys and manages containers at scale.
- **CLOUD**: Application is live in the cloud for users.

WHY LINUX?

- ✓ Open Source & Free
- ✓ Secure & Stable
- ✓ Powerful Command Line
- ✓ Automates Repetitive Tasks
- ✓ Runs Everywhere (On-prem, Cloud, Edge)

TIP

Learn Linux well, and you can work in any tech role with confidence!

REMEMBER

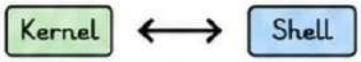
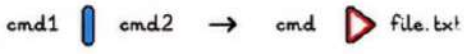
- Linux is not just an OS, it's the foundation of modern IT.
- From coding to deploying, Linux is everywhere!
- Master Linux = Career Booster ✓

> Behind every great project, there is a powerful Linux system running silently.



14. MOST ASKED LINUX INTERVIEW QUESTIONS



QUESTION	CONCISE ANSWER
1 What is Linux? 	Linux is an open-source operating system based on the Linux kernel. It is stable, secure, and widely used in servers, desktops, cloud, and devices.
2 Kernel vs Shell? 	Kernel is the core of the OS that manages hardware, memory, and processes. Shell is the command-line interface that allows users to interact with the kernel.
3 Soft Link vs Hard Link? 	• Soft Link (symbolic): Points to the path of original file. If original is deleted, link breaks. (ln -s) • Hard Link: Points to the inode of original file. If original is deleted, link still works. (ln)
4 chmod vs chown? 	• chmod (change mode): Changes file/directory permissions. Example: chmod 755 file.txt • chown (change owner): Changes ownership of a file/directory. Example: chown user:group file.txt
5 grep vs find? 	• grep: Searches for a pattern inside files. Example: grep "error" log.txt • find: Searches for files/directories by name, type, size, time, etc. Example: find /var -name "*.log"
6 Process vs Thread? 	Process is an independent program with its own memory space. Thread is the smallest unit of execution within a process and shares the same memory space.
7 Zombie Process? 	A zombie process is a terminated process whose parent has not read its exit status yet. It holds PID entry in the process table but uses no resources.
8 Cron Job? 	Cron is a time-based job scheduler in Linux. It runs commands or scripts automatically at specified time intervals. Edit with: crontab -e
9 Environment Variables? {...}	Variables that store system or user-specific values used by processes and shell. Example: $HOME, $PATH, $USER Use: echo $PATH or export VAR=value
10 Pipe (|) vs Redirection (>)? 	• Pipe (|): Sends output of one command as input to another. Example: ls -l | grep "txt" • Redirection (>): Sends output of a command to a file. Example: ls -l > out.txt

REMEMBER

- Practice commands daily.
- Understand concepts, not just syntax.
- Hands-on is the key to master Linux!





TIP

Use man <command> to read the manual and explore more examples.





"The more you practice in Linux, the more powerful you become!"



@qa_insights



15. CHEAT SHEET & ROADMAP



TOP 50 LINUX COMMANDS

#	COMMAND	PURPOSE
1	pwd	Print working directory
2	ls	List files and directories
3	cd	Change directory
4	mkdir	Create a directory
5	rmdir	Remove an empty directory
6	touch	Create an empty file
7	cp	Copy files or directories
8	mv	Move or rename files
9	rm	Remove files or directories
10	cat	Display file contents
11	less	View file content (scrollable)
12	head	Show first lines of a file
13	tail	Show last lines of a file
14	nano	Edit files in terminal
15	vi	Edit files (advanced)
16	chmod	Change file permissions
17	chown	Change file owner
18	ps	Show running processes
19	top	Monitor system processes
20	kill	Terminate a process
21	df	Disk space usage
22	du	Directory space usage
23	free	Show memory usage
24	uname	System information
25	hostname	Show or set hostname

#	COMMAND	PURPOSE
26	whoami	Show current user
27	id	Show user and group IDs
28	grep	Search text in files
29	find	Search files and directories
30	locate	Find files by name (updatedb)
31	which	Locate command path
32	echo	Display text
33	history	Show command history
34	man	Show manual of a command
35	whatis	Brief description of command
36	alias	Create command shortcut
37	tar	Archive files
38	gzip	Compress files
39	gunzip	Decompress .gz files
40	zip	Zip files
41	unzip	Unzip files
42	scp	Secure copy (remote files)
43	ssh	Secure shell login
44	ping	Check connectivity
45	netstat	Network connections
46	ss	Socket statistics
47	curl	Transfer data from URL
48	wget	Download files from web
49	crontab	Schedule tasks (cron jobs)
50	clear	Clear the terminal screen

LINUX LEARNING ROADMAP



1. Basics
- Understand OS
- File System
- Navigation

2. Commands
- Core Commands
- Text Processing
- Permissions

3. System Admin
- Users & Groups
- Processes
- Logs & Services

4. Networking
- IP, Ping, Netstat
- SSH, SCP
- Firewall Basics

5. DevOps Tools
- Git
- Docker
- CI/CD (Jenkins)

6. Orchestration
- Kubernetes
- YAML Basics
- Deploy Apps

7. Cloud
- AWS / Azure / GCP
- Linux on Cloud
- Automation

REMEMBER

- Practice daily.
- Use man <command>
- Learn by doing real tasks.
- Small steps, big progress!

TIPS

- ✓ Start with basics.
- ✓ Be consistent.
- ✓ Make your own cheat sheet.
- ✓ Practice on real projects.

BEST PRACTICES

- ✓ Use documentation.
- ✓ Automate repetitive tasks.
- ✓ Keep system updated.
- ✓ Security first!

> Linux is not just an OS, it's a skill that powers your DevOps journey!